Minnal-Q

മിന്നൽ

“Lightning-fast quantum routing for life-critical emergencies”

Severity- and Traffic-Aware Multi-Stop Ambulance Routing with QAOA

Combined project concept · QFF26 Kochi Hack · 1 November 2026

Item	Detail
Name	**Minnal** is Malayalam for lightning: pure energy, arriving in an instant. **Q** stands for the quantum (QAOA) engine. The name echoes Kerala's own superhero, Minnal Murali, and the idea that the right route has to reach a patient at the speed of lightning.
Challenge	**Challenge 1: Emergency Vehicle Routing** (Quantum Optimization · Smart Mobility)
One-line pitch	When one ambulance must reach several patients, Minnal-Q picks the visiting order that gets the most critical patients first under live traffic. The route is found with QAOA on a QUBO in Qiskit and checked against exact and greedy classical solvers.
SDG relevance	SDG 3 (Good Health: faster critical care) · SDG 11 (Sustainable Cities: resilient urban mobility) · disaster response (e.g. Kerala flood evacuation)
Built from	Patterns from three reference projects (credited in Section 9). All code is written fresh on the day, as the originality rule requires.
Stack	Python 3 · Qiskit 2.6 · qiskit-aer 0.18 · qiskit-ibm-runtime (optional) · NumPy/SciPy · Streamlit · Plotly

1. Problem statement

In a mass-casualty event such as a bus accident on NH-66, monsoon flooding or a building collapse, one ambulance often has to collect patients from several pickup points before reaching a hospital. Dispatchers usually take the nearest point first. That keeps total driving short, but it can leave a critically injured patient waiting while minor cases are picked up. Kochi's traffic varies sharply by time of day (Edappally, Vyttila, Kaloor, MG Road), so the best order at 9 a.m. can differ from the best order at 2 p.m.

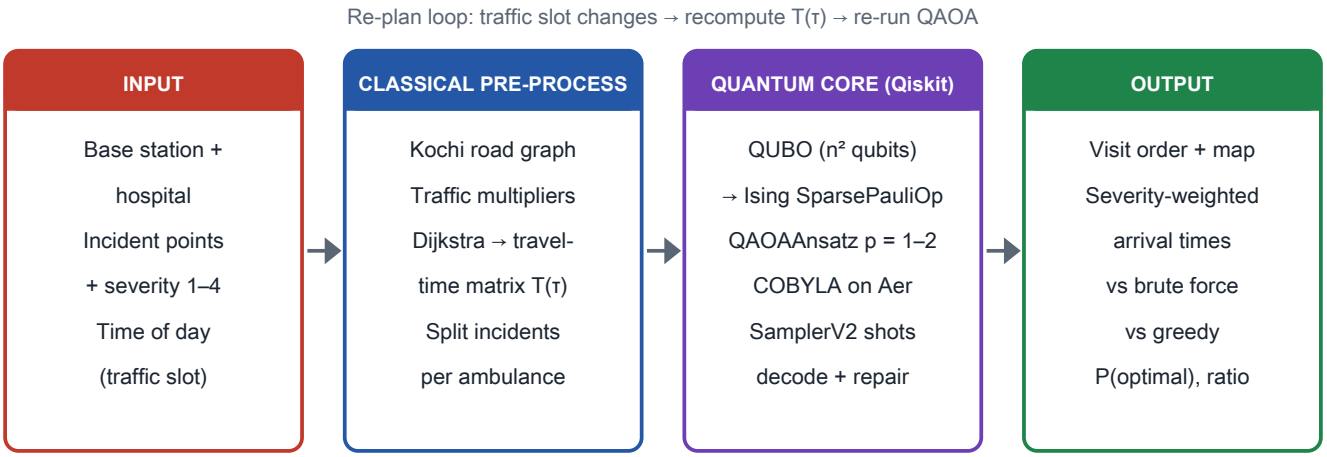
The people affected are patients whose survival depends on time to treatment (cardiac arrest, trauma, stroke), plus ambulance crews and hospital emergency departments. **What we optimize:** the order in which an ambulance visits n incident points, starting from its base and ending at a hospital. The goal is to minimize traffic-adjusted travel time while strongly preferring early visits to high-severity patients. Choosing a visiting order is a permutation problem. The number of possible routes grows as n!, and adding severity and time-dependent travel times makes simple nearest-first rules unreliable. That makes it a natural fit for a QUBO formulation solved with QAOA.

2. The combining idea: what we take from each reference

None of the 14 projects we reviewed fully covers Challenge 1 (ambulances + multi-stop routes + distance + traffic). The three strongest each cover part of it. Minnal-Q combines those parts and adds the missing piece, time-dependent traffic.

Reference	What it does well	Pattern we reuse	What Minnal-Q adds
QuRoute (QuHack 2026) Open repo ↗	Multi-stop TSP with QAOA in Qiskit; brute-force and greedy baselines; map demo	Position-encoded route QUBO; three-way comparison of the routes on a map	Ambulance framing; fixed base → hospital endpoints; traffic-dependent costs; QAOAAnsatz on Qiskit 2.6 instead of the unmaintained qiskit_algorithms; no hidden non-quantum fallback
Severity-Aware Dispatch (Qiskit Fall Fest 2026) Open repo ↗	Clean QUBO → Ising → QAOA chain; severity weights; CVaR; penalty sweep; verified against an exact solver	Exponential severity weights (1, 2, 4, 8); QUBO/Ising equivalence test; CVaR objective; penalty-tuning experiment	Severity applied to visiting order (routing), not only to which ambulance goes where
PairCover-Q / Q-Rescue (Q-HACK India 2026) Open repo ↗	Real IBM hardware runs; strong tests; honest evidence tables; polished Streamlit app	Transparency table (implemented / simulated / measured); a small hardware run; evidence page in the demo	Same honesty standard applied to a routing problem; an optional 9-qubit IBM run as a bonus
New in Minnal-Q	—	—	**Time-of-day traffic multipliers** on a Kochi junction graph, a severity-vs-time trade-off slider, and re-planning when the traffic slot changes

3. Proposed solution: Input → Processing → Output



- **Input:** ambulance base, destination hospital, 3–4 incident points with severity 1–4 (minor → critical), and the departure time slot.
- **Classical pre-processing:** a hand-built graph of about 10 Kochi junctions and hospitals. Each road's base travel time is multiplied by a congestion factor for the time slot (e.g. 1.8× on Edappally–Vyttila at peak). Dijkstra's algorithm turns this into an all-pairs travel-time matrix T(τ). With several ambulances, incidents are first split between them classically, and each route is then solved separately.
- **Quantum core:** build the QUBO, convert it to an Ising Hamiltonian, optimize QAOA parameters on the Aer simulator, sample bitstrings, decode them into a visiting order, and keep the best feasible route.
- **Output:** the route drawn on a map; per-patient arrival times; total and severity-weighted time; a comparison with the exact optimum (brute force) and with nearest-first greedy; QAOA quality metrics.

4. Quantum component (mandatory section)

4.1 Why QAOA fits

Choosing a visiting order is a binary combinatorial problem: each patient goes in exactly one route position, and the travel cost depends on which pairs of patients are consecutive. That pairwise dependence is exactly the quadratic structure a QUBO captures and QAOA is built to optimize. The quantum part therefore solves the routing decision itself; it is not a decorative circuit. This addresses the rule that "simply adding a Hadamard gate or a random quantum circuit does not qualify".

4.2 Problem → qubits mapping

Binary variable $x_{i,p} = 1$ if incident i is visited at position p ($i, p \in \{1…n\}$). The base S and hospital H are fixed endpoints, so they need no qubits. Qubit count = n^2: **n = 3 → 9 qubits** (6 possible routes), **n = 4 → 16 qubits** (24 routes). Both sizes run quickly on the Aer simulator.

4.3 QUBO cost function

$$H(x) = \mathbf{A}\cdot\Sigma_i(1 - \Sigma_p\, x_{i,p})^2 + \mathbf{A}\cdot\Sigma_p(1 - \Sigma_i\, x_{i,p})^2 \text{ (each patient once, each slot once)}$$
$$+ \Sigma_i\, T_{S,i}(\tau)\cdot x_{i,1} + \Sigma_{i\neq j}\,\Sigma_{p<n}\, T_{i,j}(\tau)\cdot x_{i,p}x_{j,p+1} + \Sigma_i\, T_{i,H}(\tau)\cdot x_{i,n} \text{ (traffic-adjusted travel time)}$$
$$+ \boldsymbol{\lambda}\cdot\Sigma_i\,\Sigma_p\, w_i\cdot(p - 1)\cdot x_{i,p} \text{ (severity: critical patients pushed earlier)}$$

- $w_i \in \{1, 2, 4, 8\}$ for severity 1–4 (exponential, following Severity-Aware Dispatch).
- λ is the severity-vs-time trade-off, exposed as a slider in the demo. λ = 0 gives a plain fastest route.
- A (constraint penalty) is set above the largest possible route cost, then tuned with a penalty sweep. We test that the lowest-energy QUBO state is always a valid route equal to the brute-force optimum.
- **Honest modelling note:** exact severity-weighted arrival time would need terms of degree higher than two, so the QUBO uses a position penalty as a quadratic proxy. Results always report the exact severity-weighted arrival time, recomputed classically for each decoded route.

4.4 Circuit and algorithm (Qiskit)

Step	Qiskit implementation
QUBO → Ising	Substitute x = (1 − Z)/2 and build a SparsePauliOp of Z and ZZ terms plus an offset. A unit test checks that the Ising energy equals the QUBO energy for every bitstring.
Ansatz	qiskit.circuit.library.QAOAAnsatz(cost_operator, reps=p), p = 1 and 2: start in the uniform superposition $H^{n^2}\|0…0\rangle$, apply alternating cost layers $e^{-i\gamma H_C}$ and mixer layers $e^{-i\beta\Sigma X}$.
Training	Aer EstimatorV2 (or StatevectorEstimator) computes the expected cost $E[H_C]$; SciPy COBYLA with 3–5 random restarts; the p = 2 run is warm-started from the best p = 1 angles. A CVaR (α = 0.2) objective is an option.
Sampling	Aer SamplerV2, 4,096 shots → bitstrings → decode the n×n position matrix → feasibility check → the lowest-cost valid route wins. Invalid samples are counted and reported, not hidden.
Backend	**Primary: qiskit_aer.AerSimulator** (named in the README). Optional bonus: one 9-qubit instance transpiled with generate_preset_pass_manager and run on an IBM backend through QiskitRuntimeService / SamplerV2, with the job ID recorded.
Hybrid loop	Classical: graph, Dijkstra, splitting incidents between ambulances, parameter optimizer, decoding. Quantum: evaluating and sampling the QAOA state. The two alternate until the optimizer converges.

5. Classical baselines and evaluation

Solver	Purpose
Brute force (all n! orders)	Exact ground truth for every instance; also validates the QUBO
Nearest-first greedy	What dispatchers do today; shows the failure case where a critical patient waits
Time-only optimum (λ = 0)	Shows the cost of adding severity: a little extra driving, much faster care for critical patients
QAOA p = 1, p = 2 (Aer)	The quantum solver under test

Metrics reported: total route time; exact severity-weighted arrival time $\Sigma w_i \cdot t_i$; critical-patient arrival time; approximation ratio vs brute force; probability of sampling the optimal route; share of valid samples; circuit depth and two-qubit gate count after transpiling. All numbers are measured on the day. None are assumed in this document.

6. Demo plan (Streamlit, live at the showcase)

- **Scenario tab:** Kochi map with base, hospital and incident pins; severity picker; time-slot selector (peak / off-peak).
- **Route tab:** three routes overlaid (greedy, time-only, Minnal-Q) with a per-patient arrival table, headed by the critical patient's arrival time.
- **Quantum lab tab:** QAOA circuit diagram, convergence curve, histogram of sampled routes with the optimum highlighted, and p = 1 vs p = 2 results.
- **Evidence tab:** transparency table, QUBO/Ising test results, penalty sweep, and the IBM job ID if a hardware run was done.
- **Demo story (≈ 3 min):** (1) greedy collects the minor case first and the critical patient waits; (2) switch the slot to peak traffic and watch the route change; (3) Minnal-Q's route matches the brute-force optimum; (4) the honesty slide.

7. Build plan for today

Time	Milestone	Done when
10:00–10:45	Kochi graph + traffic multipliers + Dijkstra matrix; brute-force and greedy solvers	Prints all routes and costs for a 3-patient scenario
10:45–12:00	QUBO builder, QUBO → Ising converter, equivalence tests	QUBO minimum = brute-force optimum; tests pass
12:00–13:30	QAOAAnsatz + COBYLA + SamplerV2 on Aer; decoding; p = 1/2; penalty sweep	QAOA returns a valid route; metrics CSV saved
13:30–14:00	Lunch	—
14:00–15:15	Streamlit app (4 tabs), map, charts	Full flow runs from one command
15:15–16:00	README, architecture diagram, screenshots; optional 9-qubit IBM run	A fresh clone runs by following the README
16:00–16:30	Code freeze, rehearse demo, submit on the portal	Submitted before the deadline

Suggested roles (team of up to 4)

Role	Owns
Quantum lead	QUBO, Ising, QAOA, experiments, optional IBM run
Classical / data lead	Kochi graph, traffic model, Dijkstra, baselines, metrics, tests
Demo lead	Streamlit app, map, charts, demo script
Docs lead	README, problem statement, diagram, transparency table, presenting

Dependencies to add today: pip install streamlit plotly networkx (Qiskit, Aer, Runtime, Matplotlib and pylatexenc are already installed).

8. Requirements compliance checklist

Kochi requirement	How Minnal-Q meets it
1. Project information	Team, project, challenge and members at the top of the README
2. Problem statement (150–300 words)	Section 1 of this document, reused in the README
3. Solution + Input → Processing → Output	Section 3 with the architecture diagram
4. Quantum component (mandatory)	Section 4: why QAOA fits, qubit mapping, QUBO, circuit, its role in the system
5. Implementation & reproducibility	.py modules and a notebook, requirements.txt, README run steps, fixed random seeds
6. Qiskit usage + backend named	QAOAAnsatz, SparsePauliOp, Aer EstimatorV2/SamplerV2; backend: AerSimulator
Rule: original work	Coded on the day; references credited in Section 9
Rule: transparency	Table in the README: implemented / simulated / measured / incomplete
Rule: authorized data	Synthetic incidents and travel times on public place names; no patient data
Extras recommended	Architecture diagram, screenshots, classical comparison, future work

Judging alignment (Phase 2)

Criterion (weight)	Where we score
MVP functionality (25%)	End-to-end app: scenario → QAOA route → comparison, in one command
Quantum implementation (25%)	A real QUBO with tested Ising equivalence, p = 1/2, CVaR, measured approximation ratios
Technical implementation (20%)	Clean modules, unit tests, reproducible seeds, optional hardware run

Criterion (weight)	Where we score
Innovation & impact (15%)	Severity + time-of-day traffic in routing, which none of the 14 reference projects combine; Kochi/Kerala context
Demo & presentation (15%)	Three-minute story built on the failure case of nearest-first dispatch

9. Transparency, limitations and credits

- **Simulated:** all main QAOA results come from AerSimulator. The traffic multipliers and incident locations are synthetic but realistic; place names are real Kochi locations.
- **Scale:** up to 4 patients per ambulance (16 qubits). Larger cases are split classically. **No quantum advantage is claimed**; the page states this is not expected.
- **Proxy:** the QUBO uses a position-based severity penalty; the exact severity-weighted metric is computed classically.
- **Credits:** patterns only, no code copied (see the reference repositories table below). Qiskit and Qiskit Aer (IBM, Apache 2.0).

Reference repositories (combined in Minnal-Q)

Project	Event	Repository link (clickable)	What we took
QuRoute	QuHack 2026	https://github.com/AnuragKannojiya/quroute	Multi-stop route QUBO, brute-force + greedy baselines, map demo
Severity-Aware Emergency Dispatch Using QAOA	IBM Qiskit Fall Fest 2026, RV University	https://github.com/shrisha77-boop/ Severity-Aware-Emergency-Dispatch-Using-QAOA	Severity weights, QUBO → Ising check, CVaR, penalty sweep
PairCover-Q / Q-Rescue	Q-HACK INDIA 2026	https://github.com/gautham-code505/PairCover-Q-	Transparency table, IBM hardware run, evidence page

Future work

- Real Kochi road network from OpenStreetMap (OSMnx) and a live traffic API.
- Kerala-flood scenario: boats and ambulances with blocked roads as hard constraints.
- Hospital capability and bed capacity in the destination choice.
- Warm-start QAOA and error mitigation on IBM hardware for larger instances.

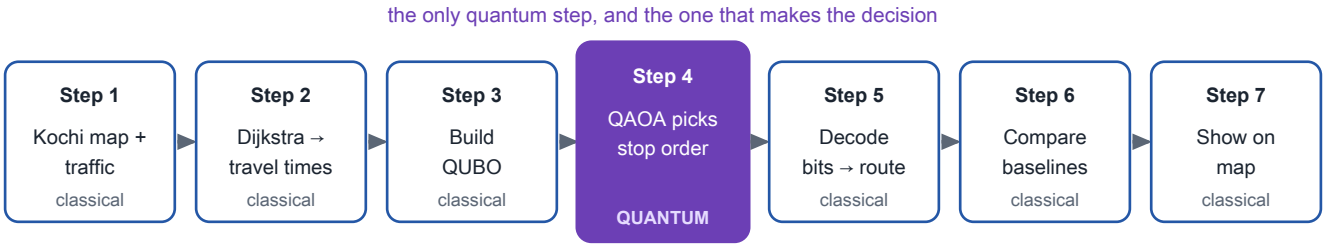
Minnal-Q

മിന്നൽ

Where and why quantum is used

The short answer: quantum is used in **one place**: choosing the **order in which the ambulance visits patients**. A QAOA circuit in Qiskit encodes every possible visiting order into qubits and is trained so that the safest, fastest route becomes the most likely measurement. Everything around it (map, traffic, checking, display) is classical. **Remove the quantum step and Minnal-Q has no route.**

1. WHERE quantum sits in the pipeline



Steps 1–3 prepare the quantum input · Steps 5–7 read and check the quantum output

Step	What happens	Type	Tool
1	Kochi junctions and hospitals; road times × time-of-day traffic factor	Classical	Python
2	Shortest travel time between every pair of points	Classical	Dijkstra / NetworkX
3	Turn the routing problem into a QUBO cost formula	Classical (prepares quantum input)	NumPy
4	**QAOA circuit finds the best visiting order**	**QUANTUM**	**Qiskit QAOAAnsatz, SparsePauliOp, AerSimulator**
5	Turn measured bitstrings into routes; drop invalid ones	Classical	Python
6	Check against the exact best route and greedy	Classical	Python
7	Draw routes, charts and evidence	Classical	Streamlit + Plotly

2. WHY quantum fits this problem

Reason	Explanation
Routing is a yes/no problem	"Is patient A at stop 1?" is a yes/no question, and a qubit holds exactly one yes/no answer (0 or 1). A full route is a pattern of these answers, so it maps directly onto qubits.
The costs come in pairs	Travel time depends on which two patients are next to each other in the route. Pairwise costs are exactly the form a QUBO captures, and a QUBO is exactly what QAOA is built to solve. There is no forced fit.
The number of routes explodes	The number of possible orders grows as n! (n factorial), as the table below shows. That growth is why routing is a classic benchmark for quantum optimization.
QAOA is built for today's machines	QAOA uses short, shallow circuits and a classical helper. It runs well on simulators and is designed for current noisy quantum computers, including IBM hardware.
It is meaningful, not decorative	The rules say "adding a Hadamard gate or a random quantum circuit does not qualify". In Minnal-Q the circuit is the decision-maker: its output is the route.
It is ready to scale	As quantum hardware grows, the same code can handle more stops. Only the backend changes.

How fast the number of routes grows

Patients (n)	Possible orders (n!)	Qubits needed (n^2)	Where it runs
3	6	9	Live demo on AerSimulator (fast); IBM hardware bonus
4	24	16	AerSimulator experiment
6	720	36	Too big to simulate exactly on a laptop
10	3,628,800	100	Needs future hardware; brute force is already slow
15	about 1.3 trillion	225	Brute force is impractical

Honesty note: at demo size (3–4 patients) a classical computer is still faster. We do **not** claim quantum advantage, and the event page says it is not expected. We show a correct, verified quantum pipeline for a problem that becomes very hard as it grows.

3. HOW the quantum step works

3.1 Every qubit is a yes/no question

Three patients (A = critical, B = medium, C = minor) and three stops need 3 × 3 = **9 qubits**:

	Stop 1	Stop 2	Stop 3
Patient A (critical)	q0	q1	q2
Patient B (medium)	q3	q4	q5
Patient C (minor)	q6	q7	q8

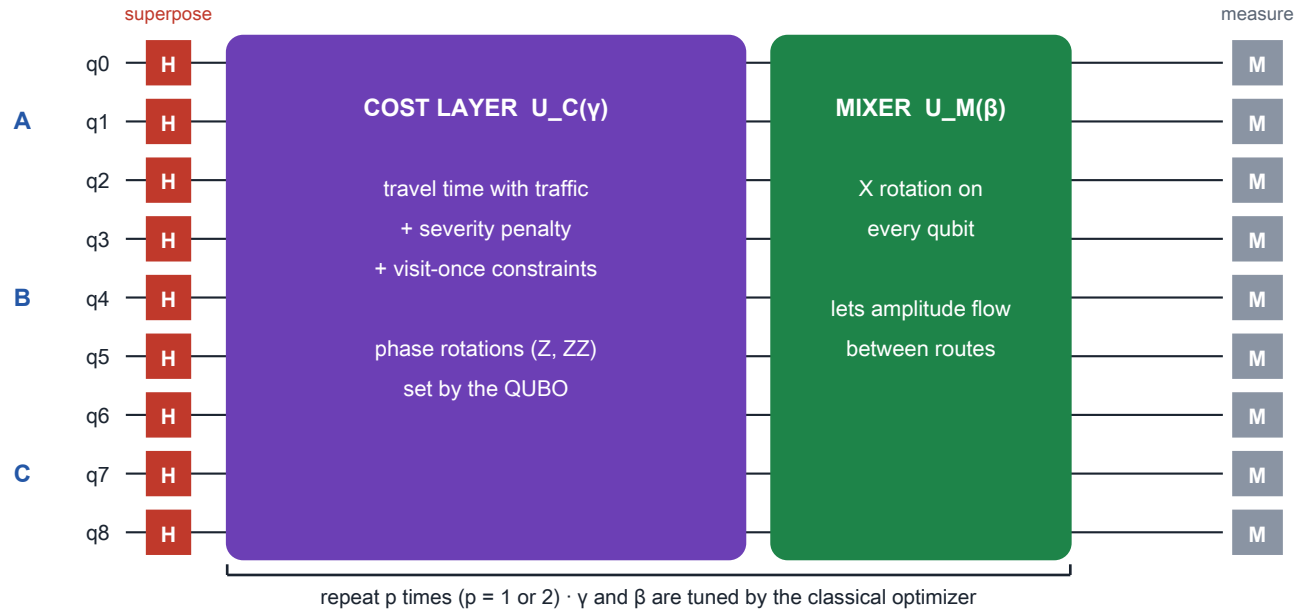
Example: the route **A → B → C** is the bit pattern **100 · 010 · 001** (q0 to q8). Qiskit prints bitstrings in reverse order (q8 first); the decoder handles that.

3.2 The cost formula becomes the circuit's energy

Part of the cost	What it does	Effect
Travel time with traffic	Adds the drive time between consecutive stops at the chosen time of day	Prefers short routes
Severity penalty	Adds cost for visiting a high-severity patient late (weights 1, 2, 4, 8)	Critical patients first
Visit-once constraints	Large cost if a patient is skipped or visited twice	Only valid routes win

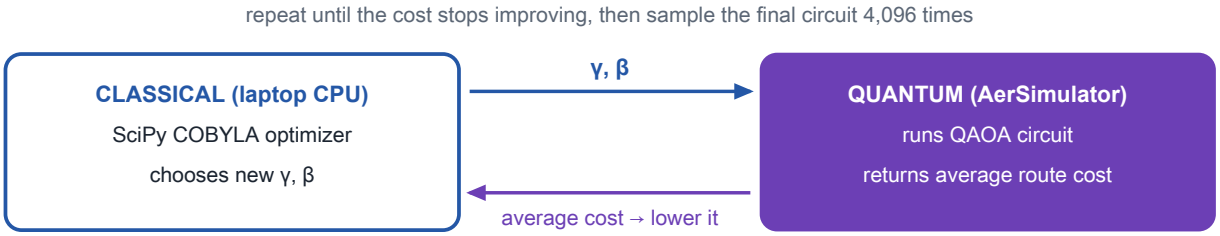
Qiskit converts this formula into a quantum operator (SparsePauliOp). **The best route is the lowest-energy state**, and QAOA searches for it.

3.3 The QAOA circuit



- **H gates (superpose):** start all 9 qubits in an equal mix of every possible bit pattern.
- **Cost layer (γ):** phase rotations that encode every route's cost into the quantum state.
- **Mixer layer (β):** moves probability between routes so that cheaper routes become more likely.
- **Measure:** each run returns one bit pattern, which is one candidate route.

3.4 The hybrid loop (quantum and classical working together)



The classical optimizer never sees the routes directly. It only adjusts the two angles γ and β so that the quantum circuit's average route cost goes down. This is the "hybrid" design the requirements ask teams to explain.

3.5 Measurement gives the answer

Measured bits (q0 to q8)	Decoded route	Valid?	Meaning
100 · 010 · 001	A → B → C	Yes	Critical first: the kind of route Minnal-Q should favour
100 · 001 · 010	A → C → B	Yes	Critical first, different order for the other two
001 · 010 · 100	C → B → A	Yes	Nearest-first style: critical patient last (high cost)
110 · 000 · 001	A twice, B missing	No	Broken route, filtered out by the decoder

Illustrative examples only. Real probabilities are measured on the day from 4,096 shots and shown in the demo's Quantum Lab tab.

4. Qiskit components used

Qiskit component	Role in Minnal-Q
qiskit.quantum_info.SparsePauliOp	Holds the route-cost formula as a quantum operator (Z and ZZ terms)
qiskit.circuit.library.QAOAAnsatz	Builds the QAOA circuit: superposition, cost layer, mixer, p repetitions
qiskit_aer.AerSimulator / EstimatorV2	Simulates the circuit and returns the average cost while training
qiskit_aer SamplerV2	Runs the trained circuit 4,096 times to collect candidate routes
generate_preset_pass_manager	Compiles the circuit for a real device (optional hardware run)
qiskit_ibm_runtime (optional)	Sends the 9-qubit version to an IBM quantum computer; the job ID goes in the README

5. Questions judges may ask

Question	Answer
Is the quantum part meaningful?	Yes. The QAOA circuit makes the routing decision. Without it there is no route; it is not a decorative circuit.
Why not just use brute force?	We do use it, as the answer key that proves QAOA is correct. Brute force breaks down as patients increase (n! growth). QAOA is the approach built to scale with quantum hardware.
Is it faster than classical?	Not at demo size, and we say so openly. The event does not expect quantum advantage. We show a correct, tested pipeline.
Why QAOA and not Grover or VQE?	Our problem is a cost-minimization QUBO. QAOA is designed exactly for that and runs well on today's noisy devices. Grover needs much deeper circuits, and VQE is aimed at chemistry-style problems.
What about real hardware?	Main results come from AerSimulator (allowed by the rules and documented). As a bonus we run the 9-qubit version on IBM hardware and report the job ID, including the effect of noise.
How do you know the answer is right?	Every QAOA route is checked against the exact best route found by brute force. We also test that the QUBO's lowest-energy state equals that exact answer.

6. The 30-second explanation for judges

"Quantum decides the most important thing in Minnal-Q: the order in which the ambulance reaches patients. We turn every possible route into 9 qubits. The QAOA circuit is trained, with a classical optimizer, to make the route that gets the critical patient first, through Kochi traffic, the most likely result. When we measure it, that route comes out on top, and we check it against the exact answer. Minnal means lightning, and that's how fast the right route has to arrive."

Minnal-Q

മിന്നൽ

Real-life use cases for quantum emergency routing

The core question in every scenario: one vehicle, several people who need help, roads that change with traffic or disaster, so in what order do we go? Minnal-Q answers it with a QAOA circuit that weighs travel time, traffic and how critical each person is. The scenarios below show where this decision comes up in real life, especially in Kerala.

Readiness labels used in this document

Label	Meaning
DEMO TODAY	Fits the hackathon build: 3–4 stops, 9–16 qubits, runs on AerSimulator today
NEAR FUTURE	Same engine plus extra data (real road maps, blocked roads, time windows); still small per vehicle
FUTURE HARDWARE	Needs many more qubits or splitting into many small sub-problems; depends on larger quantum computers

We use these labels to stay honest: the hackathon demo proves the method on small cases, and the larger scenarios show where it leads.

Use-case scenarios

1. Mass-casualty road accident	DEMO TODAY
Situation	A bus overturns on NH-66 near Edappally. One ambulance arrives first; three or four injured people are spread along the road and nearby side lanes, with different injury levels.
The decision	Who to reach first and in what order, before taking everyone to the trauma centre.
Quantum role	QAOA orders the stops so the critical patient (severity 4) is reached first, using traffic-adjusted travel times. 3 victims → 9 qubits.
Inputs → Output	Victim locations + triage severity + time of day → visiting order and arrival time per patient
Who benefits	Critical trauma patients (the "golden hour"), ambulance crews, hospital emergency teams

2. Flood rescue	NEAR FUTURE
Situation	Heavy monsoon flooding, as in the 2018 Kerala floods. A rescue boat or high-clearance vehicle must pick up stranded people from several homes and bring them to a relief camp or hospital.
The decision	Pickup order when some routes are under water and people have different medical needs (elderly, pregnant, injured).
Quantum role	Same QUBO; flooded roads get a very high travel cost so QAOA avoids them. Severity weights give priority to the most vulnerable.
Inputs → Output	Rescue requests + flooded-road map + vulnerability level → pickup order and safe route
Who benefits	Stranded families, disaster response teams, district emergency operation centres

3. Landslide response	NEAR FUTURE
Situation	Landslides in a hilly district, like Wayanad in 2024, cut roads and isolate several sites. Teams must reach multiple locations as roads reopen.
The decision	Which sites a vehicle visits, in what order, as roads open and close during the day.
Quantum role	Re-run QAOA every time the road status changes (the "re-plan loop"). Blocked roads are high-cost edges; sites with more casualties carry higher weight.
Inputs → Output	Site list + casualty estimates + live road status → updated route for each vehicle
Who benefits	Survivors in isolated areas, NDRF and fire-and-rescue teams, hospitals receiving patients

4. Festival and pilgrimage crowds	DEMO TODAY
Situation	Large gatherings such as Thrissur Pooram or the Sabarimala season. Medical teams handle several emergencies at once (heat stroke, falls, cardiac events) inside heavily congested areas.
The decision	Order of patient visits when congestion changes by the hour.
Quantum role	Time-of-day traffic factors become crowd-density factors. QAOA picks the order that reaches the critical cases first through the least crowded paths.
Inputs → Output	Emergency calls + crowd-density map + severity → visiting order for each medical unit
Who benefits	Festival visitors and pilgrims, event medical teams, police control rooms

5. Blood and organ transport	NEAR FUTURE
Situation	A vehicle must collect rare blood units from two or three blood banks, or carry a donor organ, and reach the hospital within a strict time limit. Kerala has used police-cleared "green corridors" for organ transport.
The decision	Pickup order that keeps every item within its safe time window.
Quantum role	Time-critical items get high weight; time limits are added as penalty terms in the QUBO.
Inputs → Output	Pickup points + time limits + traffic → order and timing that keeps every item viable
Who benefits	Transplant and surgery patients, blood banks, hospitals

6. Home-care and palliative visits	NEAR FUTURE
Situation	Kerala's community palliative-care network sends nurses or a doctor to several elderly or bed-ridden patients' homes in a day; some calls are urgent.
The decision	Visiting order that handles urgent patients early while keeping total travel low.
Quantum role	Same engine: severity becomes urgency; the trade-off slider balances travel time against urgency.
Inputs → Output	Patient list + urgency + traffic → daily visiting route
Who benefits	Elderly and palliative patients, community nurses, primary health centres

7. Oxygen and medicine supply in a health crisis	NEAR FUTURE
Situation	During a surge like COVID-19 in 2021, a vehicle delivers oxygen cylinders or critical medicines to several hospitals and homes with different levels of need.
The decision	Delivery order when some patients cannot wait.
Quantum role	Each stop's need sets its weight; QAOA orders deliveries so the most urgent stops come first.
Inputs → Output	Delivery requests + need level + traffic → delivery order
Who benefits	Patients at home and in hospital, health departments, supply teams

8. Inter-hospital ICU transfers	DEMO TODAY
Situation	A critical-care ambulance moves several stable-but-serious patients between hospitals (for scans, surgery or free ICU beds) in one trip.
The decision	Order of hospital stops that meets the most urgent transfers first.
Quantum role	Hospitals become stops; transfer urgency becomes severity. Small enough for today's simulator.
Inputs → Output	Transfer requests + urgency + traffic → stop order
Who benefits	ICU patients, hospital networks, bed-management teams

9. City-wide ambulance fleet	FUTURE HARDWARE
Situation	A whole city's emergency ambulance service (for example Kerala's 108 network) handling dozens of calls and vehicles at once.
The decision	Which ambulance serves which calls, and in what order, for the whole fleet together.
Quantum role	Too large for today's devices in one piece. The path forward is to split calls between ambulances classically, then run one small QAOA per ambulance, which Minnal-Q already does. Bigger quantum computers could solve larger pieces together.
Inputs → Output	All live calls + all ambulances + traffic → fleet-wide routes
Who benefits	An entire city's population, emergency control rooms, health planners

Summary of all scenarios

#	Scenario	Stops per vehicle	Qubits	Readiness
1	Mass-casualty road accident	3–4	9–16	**Demo today**
2	Flood rescue	3–5	9–25	**Near future**
3	Landslide response	3–5	9–25	**Near future**
4	Festival and pilgrimage crowds	3–4	9–16	**Demo today**
5	Blood and organ transport	2–4	4–16	**Near future**
6	Home-care and palliative visits	3–5	9–25	**Near future**
7	Oxygen and medicine supply	3–5	9–25	**Near future**
8	Inter-hospital ICU transfers	2–4	4–16	**Demo today**
9	City-wide ambulance fleet	Dozens	Hundreds+	**Future hardware**

Qubits = (stops per vehicle)². Above about 25–30 qubits, exact simulation on a laptop becomes slow; those cases need quantum hardware or more splitting into smaller pieces.

How one quantum engine adapts to every scenario

The QAOA circuit and the Qiskit code stay the same. Only the numbers fed into the QUBO change:

What changes	In the QUBO	Example scenarios
Traffic or crowds	Travel-time multipliers on each road	City accidents, festivals
Blocked roads	Very high cost on that road, so it is never chosen	Floods, landslides
How critical a person is	Severity weight (1, 2, 4, 8)	All emergency scenarios
Time limits	Extra penalty if a stop is reached too late	Blood and organ transport
Changing conditions	Rebuild the QUBO and re-run QAOA (re-plan loop)	Landslides, live traffic
More vehicles	Split stops between vehicles classically, then one QAOA per vehicle	City-wide fleet

Beyond emergencies: the same idea elsewhere

Choosing the best order or allocation under pairwise costs is a common pattern, so the Minnal-Q approach (QUBO + QAOA + classical checks) carries over to other areas:

- **Logistics and last-mile delivery:** courier and grocery routes in busy cities.
- **EV charging:** deciding when and where vehicles charge (Challenge 4 at this event).
- **Energy:** choosing which power sources meet demand at the lowest cost (Challenge 3).
- **School and public buses:** stop order and timing for many routes.
- **Waste collection and street maintenance:** visit order for municipal vehicles.

Honest reality check

- Today, classical computers are faster for every scenario at the sizes we can run. We do **not** claim quantum advantage.
- Our demo proves the method works correctly on small, real-sized pieces (3–4 stops per vehicle), checked against the exact best answer.
- Real deployment would need real road data, live traffic feeds, integration with control rooms, and larger quantum hardware for the biggest cases.
- The value of Minnal-Q today is a **tested, quantum-ready pipeline** for decisions where every minute counts.

Pitch line

"Whether it's a bus crash on NH-66, a flood in Kuttanad or a crowd at Thrissur Pooram, the question is always the same: one vehicle, many people, which order? Minnal-Q answers it with a quantum circuit that puts the most critical person first. Today it runs on a simulator for one ambulance. Tomorrow, on bigger quantum computers, it can route a whole city's emergency fleet."